Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Процессы и профессиональная практика программной инженерии»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Алешов Е.В.

ПРАКТИЧЕСКАЯ РАБОТА 5. КОНФИГУРАЦИОННОЕ УПРАВЛЕНИЕ

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной работы является изучение принципов и методов конфигурационного управления программными продуктами, освоение инструментов контроля версий, организации ветвления и слияния изменений, автоматизации сборки программного обеспечения и формирования базовых версий программных активов.

1.2. Задачи

- Раскрыть понятие конфигурационного управления и определить его роль в жизненном цикле программного обеспечения.
- Изучить единицы конфигурационного управления и их основные характеристики.
- Рассмотреть принципы управления версиями файлов и составных конфигурационных объектов.
- Изучить особенности централизованных и распределённых систем контроля версий.
- Освоить базовые принципы работы с Git, включая рабочий каталог, индекс, фиксацию изменений и ветвление.
- Рассмотреть назначение автоматизированной сборки и непрерывной интеграции программных проектов.
- Определить роль базовой линии (baseline) в обеспечении целостности программного продукта.
- Изучить основные функции управления конфигурацией и релизами программного обеспечения.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ КОНФИГУРАЦИОННОГО УПРАВЛЕНИЯ

2.1. Понятие и назначение конфигурационного управления

Конфигурационное управление представляет собой совокупность процессов, обеспечивающих идентификацию, хранение, отслеживание изменений и поддержание согласованного состояния программных активов проекта. Основными направлениями конфигурационного управления являются управление версиями и управление сборками. Управление версиями позволяет сохранять историю изменений файлов, восстанавливать предыдущие состояния и координировать совместную работу разработчиков. Управление сборками обеспечивает автоматизированное преобразование исходного кода в исполняемые модули и готовые пакеты программного обеспечения. Комплексное применение этих процессов повышает воспроизводимость разработки, упрощает интеграцию компонентов и снижает риск возникновения ошибок.

2.2. Единицы конфигурационного управления

- Исходный код программного обеспечения — файлы с программной логикой, конфигурациями и вспомогательными компонентами.
- Проектная документация — архитектурные описания, технические задания, спецификации и схемы.
- Пользовательская документация — инструкции, справочные материалы и руководства по эксплуатации.
- Тестовые пакеты — наборы тестов, сценарии проверки и тестовые данные.
- Инсталляционные пакеты — подготовленные для установки дистрибутивы и исполняемые компоненты.
- Тестовые отчёты — результаты проверок, сведения об обнаруженных дефектах и подтверждения качества.
- Сценарии сборки и конфигурационные файлы — средства автоматизации компиляции и настройки окружения.

2.3. Основные характеристики единиц конфигурационного управления

Каждая единица конфигурационного управления характеризуется структурой, ответственными лицами, установленными правилами изменения и процедурами контроля целостности. Структура определяет состав файлов, взаимосвязи и зависимости между ними. Ответственные лица обеспечивают разработку, сопровождение и согласование изменений. Практика конфигурационного управления регламентирует порядок внесения изменений, правила именования, комментирования, фиксации и публикации версий. Контроль целостности позволяет проверять согласованность компонентов, например посредством компиляции, автоматического тестирования и проверки зависимостей. Единицы конфигурационного управления могут образовывать иерархию: файлы объединяются в компоненты, компоненты — в проекты, а проекты — в составные программные продукты.

2.4. Управление версиями и ветвление проекта

Управление версиями обеспечивает сохранение истории изменений программных активов и возможность восстановления ранее зафиксированного состояния. Для одновременной разработки новых функций, исправления ошибок и подготовки выпусков используются ветки. Ветка представляет собой независимое направление развития проекта, позволяющее изолировать изменения до момента их интеграции. Типичная организация репозитория включает основную ветку master или main, ветки релизов, ветки исправлений и экспериментальные ветки. Ветка релиза предназначена для стабилизации готовящегося выпуска, ветка исправления создаётся для устранения дефектов определённой версии, а экспериментальная ветка используется для проверки технических решений. После завершения работ изменения могут объединяться с другими ветками посредством слияния.

2.5. Централизованные и распределённые системы контроля версий

Система контроля версий (СКВ) регистрирует изменения файлов и позволяет сравнивать, восстанавливать и анализировать различные версии программного продукта. В централизованных системах, например Subversion и Perforce, основная база версий хранится на центральном сервере, к которому обращаются разработчики. Такой подход упрощает централизованное администрирование и контроль изменений, но создаёт зависимость от доступности сервера. В распределённых системах, например Git и Mercurial, каждый разработчик располагает локальной копией репозитория с историей изменений. Это позволяет выполнять большинство операций без подключения к сети и повышает устойчивость к отказам центрального сервера. Выбор типа СКВ определяется размером команды, требованиями к совместной работе, инфраструктурой и особенностями проекта.

III. ИНСТРУМЕНТЫ И МЕТОДЫ КОНФИГУРАЦИОННОГО УПРАВЛЕНИЯ

3.1. Основы работы с Git

Git является распределённой системой контроля версий, которая сохраняет историю состояний проекта и использует контрольные суммы для идентификации содержимого объектов. Основными областями работы являются рабочий каталог, область подготовленных файлов (staging area) и каталог Git, содержащий базу объектов и метаданные репозитория. В рабочем каталоге разработчик редактирует файлы, затем выбирает изменения для следующей фиксации и сохраняет их командой commit. Изменения могут передаваться в удалённый репозиторий посредством push, а новые данные из него могут получаться посредством fetch или pull. Git поддерживает локальную работу, сравнение версий, восстановление состояния файлов, создание веток и слияние изменений.

3.2. Основные операции Git

- git init — создание нового локального репозитория.
- git clone — создание локальной копии существующего репозитория.
- git status — просмотр состояния рабочего каталога и области подготовленных файлов.
- git add — добавление выбранных изменений в область подготовки.
- git commit — фиксация подготовленных изменений в истории репозитория.
- git log — просмотр истории фиксаций.
- git diff — сравнение изменений между состояниями файлов.
- git branch — просмотр и создание веток.
- git switch — переключение между ветками.
- git merge — объединение изменений из одной ветки с другой.
- git push — отправка локальных фиксаций в удалённый репозиторий.
- git pull — получение изменений из удалённого репозитория и их интеграция в текущую ветку.

3.3. Организация процесса управления версиями

Эффективное управление версиями требует единых правил командной работы. Каждый разработчик должен создавать логически завершённые фиксации, сопровождать их понятными сообщениями и регулярно синхронизировать локальную ветку с общим репозиторием. Перед слиянием необходимо проверять совместимость изменений, разрешать конфликты и запускать тесты. Для основной ветки рекомендуется устанавливать ограничения на прямую запись, использовать проверку изменений посредством запросов на слияние и автоматически запускать сборку. Конфиденциальные данные, пароли, токены доступа и локальные артефакты сборки не следует включать в репозиторий без обоснованной необходимости.

3.4. Управление сборками и непрерывная интеграция

Управление сборками представляет собой автоматизированный процесс преобразования исходного кода в исполняемые модули, библиотеки и установочные пакеты с учётом зависимостей и параметров компиляции. Для автоматизации применяются сценарии сборки и специализированные инструменты, позволяющие повторять процедуру на разных компьютерах и в серверной среде. Непрерывная интеграция (Continuous Integration, CI) предполагает регулярное объединение изменений разработчиков с последующей автоматической сборкой и запуском тестов. Такая практика помогает обнаруживать ошибки интеграции на ранних этапах, проверять совместимость компонентов и поддерживать актуальное состояние проекта. При необходимости процесс расширяется автоматической подготовкой дистрибутивов и публикацией результатов сборки.

3.5. Базовая линия (baseline)

Baseline — это зафиксированная базовая версия программного продукта или другого разрабатываемого актива, которая прошла установленные процедуры проверки и принята в качестве точки отсчёта для дальнейших изменений. Базовая линия может включать исходный код, документацию, конфигурационные файлы, тесты и результаты сборки. Её назначение состоит в обеспечении согласованности работы команды, воспроизводимости результатов и возможности сравнивать последующие изменения с утверждённым состоянием. Для программного проекта базовая линия обычно сопровождается идентификатором версии, контрольной фиксацией в системе контроля версий и подтверждением успешного прохождения необходимых проверок.

IV. АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ ПРОЦЕССА КОНФИГУРАЦИОННОГО УПРАВЛЕНИЯ

4.1. Значение конфигурационного управления для программного проекта

- Конфигурационное управление обеспечивает прослеживаемость изменений и помогает определить, когда, кем и по какой причине была изменена конкретная часть программного продукта.
- Системы контроля версий позволяют нескольким разработчикам работать параллельно, сохраняя историю изменений и снижая риск потери результатов работы.
- Ветвление позволяет изолировать разработку новых функций, исправление дефектов и подготовку релизов, не нарушая стабильность основной версии.
- Автоматизированная сборка выявляет ошибки компиляции, проблемы зависимостей и несогласованность отдельных компонентов программной системы.
- Непрерывная интеграция обеспечивает регулярную проверку совместимости изменений и сокращает время между появлением ошибки и её обнаружением.
- Базовая линия создаёт воспроизводимую точку отсчёта, необходимую для тестирования, передачи продукта заказчику и подготовки выпусков.
- Отсутствие единых правил фиксации, именования версий и интеграции может приводить к конфликтам, расхождению рабочих копий и невозможности воспроизвести выпуск.

4.2. Практическая модель организации работы

В качестве практической модели конфигурационного управления предлагается использовать Git для хранения исходного кода и документации, основную ветку main для проверенного состояния проекта, отдельные ветки для новых функций и исправлений, а также автоматизированный процесс сборки и тестирования. Разработчик создаёт ветку, вносит изменения, проверяет их локально, фиксирует результат и отправляет ветку в общий репозиторий. После проверки изменений выполняется слияние с основной веткой. Система непрерывной интеграции собирает проект и запускает автоматические тесты. При успешном завершении проверок создаётся идентифицируемая базовая версия, которая может использоваться для подготовки релиза. Для критичных проектов дополнительно предусматриваются резервное копирование, разграничение доступа и документирование изменений.

4.3. Управление релизами программного продукта

Управление релизами представляет собой процесс планирования, подготовки, проверки и выпуска определённых версий программного продукта. По масштабу изменения условно подразделяются на крупные функциональные выпуски, минорные выпуски с расширением возможностей и исправительные выпуски, направленные преимущественно на устранение дефектов. В распространённой схеме семантического версионирования используются обозначения MAJOR.MINOR.PATCH, где увеличение MAJOR связано с несовместимыми изменениями интерфейсов, MINOR — с добавлением совместимой функциональности, PATCH — с исправлениями, сохраняющими совместимость. По способу реализации можно выделить плановые функциональные релизы, исправительные релизы, предварительные версии для тестирования и экспериментальные выпуски. Процесс управления релизами включает определение состава выпуска, фиксацию версии, проверку качества, подготовку примечаний к выпуску, формирование дистрибутива, публикацию и сопровождение выпущенной версии.

V. ВЫВОДЫ

В ходе изучения темы рассмотрены назначение и основные принципы конфигурационного управления программными продуктами, характеристики единиц конфигурационного управления, методы контроля версий, особенности централизованных и распределённых систем, а также основы работы с Git. Установлено, что управление версиями обеспечивает сохранение истории изменений и согласованную работу разработчиков, а автоматизированная сборка и непрерывная интеграция позволяют своевременно обнаруживать ошибки и поддерживать целостность программного продукта. Базовая линия служит опорной версией для разработки, тестирования и подготовки релизов. Эффективная организация конфигурационного управления предполагает применение единых правил работы с репозиторием, понятных сообщений фиксации, контролируемого ветвления, автоматизированных проверок и идентифицируемых выпусков. Использование этих практик повышает надёжность разработки, воспроизводимость сборок, прозрачность командной работы и качество конечного программного продукта.

VI. КОНТРОЛЬНЫЕ ВОПРОСЫ И ОТВЕТЫ

6.1. Назначение конфигурационного управления программными продуктами

Конфигурационное управление предназначено для идентификации, хранения, отслеживания и контролируемого изменения программных активов. Оно обеспечивает сохранность истории версий, согласованность компонентов, воспроизводимость сборок и возможность восстановления предыдущего состояния проекта.

6.2. Единицы конфигурационного управления

К единицам конфигурационного управления относятся исходный код, проектная и пользовательская документация, тестовые пакеты, конфигурационные файлы, сценарии сборки, инсталляционные пакеты и тестовые отчёты. Каждая единица должна иметь определённый состав и правила сопровождения.

6.3. Основные характеристики единицы конфигурационного управления

Основными характеристиками являются структура, ответственное лицо или группа, практика конфигурационного управления и процедура контроля целостности. Эти характеристики определяют состав элемента, порядок внесения изменений, распределение ответственности и способы проверки корректности.

6.4. Управление версиями файлов

Управление версиями заключается в регистрации изменений файлов, сохранении истории, сравнении состояний, восстановлении предыдущих версий и координации совместной работы разработчиков. Для этого применяются системы контроля версий, например Git и Subversion.

6.5. Управление сборками программной системы

Управление сборками обеспечивает автоматизированную компиляцию исходного кода, разрешение зависимостей, формирование исполняемых модулей и создание дистрибутивов. Автоматизация делает сборку воспроизводимой и позволяет интегрировать её с тестированием и непрерывной интеграцией.

6.6. Понятие базовой линии (baseline)

Базовая линия — это идентифицированная и проверенная версия программного продукта или его компонентов, используемая в качестве точки отсчёта для дальнейшей разработки. Она помогает обеспечить согласованность команды и воспроизводимость результатов.

6.7. Управление конфигурацией программной системы

Управление конфигурацией программной системы охватывает определение конфигурационных единиц, ведение версий, контроль изменений, организацию сборок, проверку целостности и фиксацию базовых версий. Эти действия выполняются на протяжении жизненного цикла программного продукта.

6.8. Функции процесса управления конфигурацией

К основным функциям относятся идентификация конфигурационных единиц, управление версиями, контроль и согласование изменений, учёт состояния конфигурации, аудит соответствия установленным требованиям, обеспечение целостности и поддержка воспроизводимости сборок.

6.9. Управление релизами программного продукта

Управление релизами включает планирование выпуска, определение состава изменений, подготовку версии, тестирование, сборку дистрибутива, документирование, публикацию и сопровождение выпущенного продукта. Оно обеспечивает выпуск проверенных и идентифицируемых версий.

6.10. Классификация релизов по масштабу

По масштабу изменений релизы можно разделить на крупные, содержащие существенные функциональные или архитектурные изменения; минорные, добавляющие совместимую функциональность; и исправительные, предназначенные для устранения дефектов. В семантическом версионировании этим категориям соответствуют компоненты MAJOR, MINOR и PATCH.

6.11. Классификация релизов по способу реализации

По способу реализации выделяют плановые функциональные релизы, исправительные релизы, предварительные версии для тестирования, экспериментальные сборки и версии с долгосрочной поддержкой. Конкретная классификация зависит от принятой в организации модели выпуска и сопровождения продукта.

6.12. Функции процесса управления релизами

Функции управления релизами включают планирование сроков выпуска, формирование состава версии, контроль готовности изменений, проверку качества, идентификацию и маркировку версии, создание дистрибутива, подготовку документации, публикацию, развёртывание и анализ проблем после выпуска.